

Database 2

JPA exercise

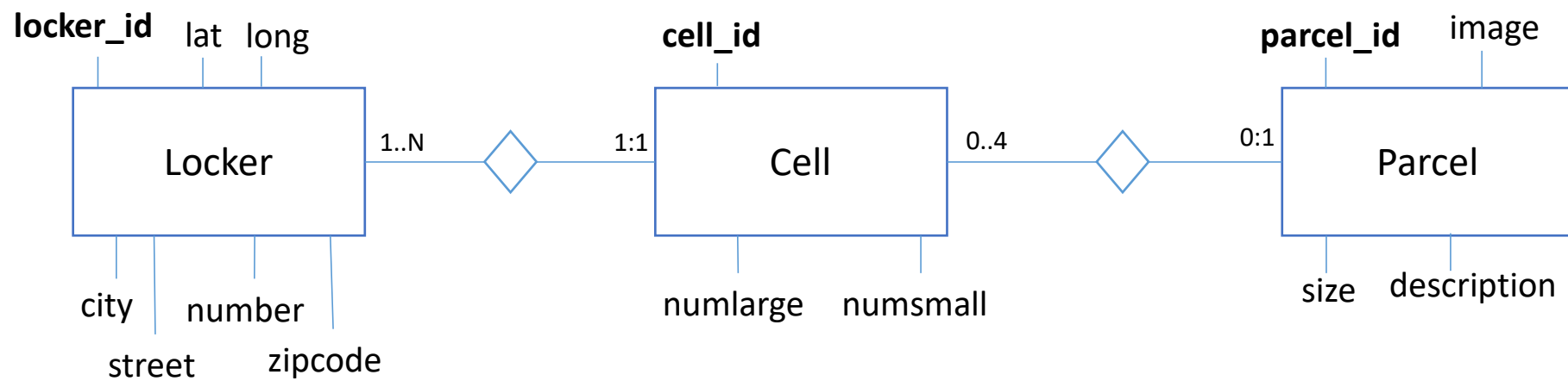
Exam on July 15 2022

Solutions

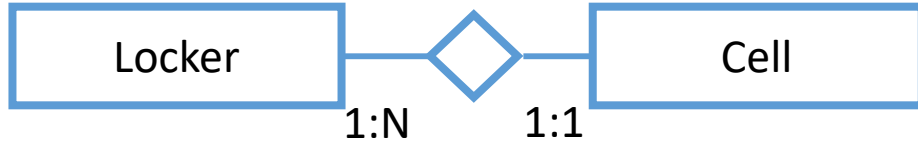
Specifications

Given the database of the parcel delivery company of exercise C, an application permits the employee to access the list of the lockers in one city, access the list of the cells of the locker, check the status of occupancy and the details of the parcels contained. Lockers are in the order of thousands and cells per lockers are in the order of tens.

Draw the Entity-relationship model corresponding to the relational model, show the JPA entities (with all their attributes) that map the domain objects of the relational model, taking into account the above-mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.



Relationship Locker-Cell

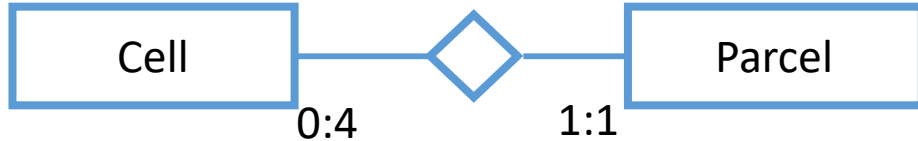


- Locker → Cell
- @OneToMany
 - Necessary to browse the cells of a locker
 - Owner = cell
 - FetchType is EAGER (Cells per Locker are in the order of tens)
 - Persist can be cascaded, and Remove must be cascaded

Cell → Locker
@ManyToOne

- Not necessary but mapped for consistency
- FetchType is EAGER, only one locker for cell
- No cascade is needed

Relationship Cell-parcel



- Cell → Parcel
@OneToMany
 - needed to show all the Parcels in Cell
 - Owner = Parcel
 - FetchType can be EAGER (parcel per cell go from 0 to 4)
 - Remove can be cascaded
- Parcel → Cell
@ManyToOne
 - Not necessary but mapped for consistency
 - FetchType can be EAGER (one cell per parcel)
 - No cascade is needed

Entity Locker

```
@Entity
public class Locker implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private long lockerId;
    private String city;
    private String street;
    private int number;
    private int zipcode;
    private float lat;
    private float lon;

    @OneToMany(mappedBy="locker", fetch = FetchType.EAGER,
               cascade = {CascadeType.PERSIST, CascadeType.REMOVE})
    private List<Cell> cells;

    // getters setters and constructors
    // . . .

}
```

Entity Cell

```
@Entity
public class Cell implements Serializable {
    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private long cellId;
    private int numlarge;
    private int numsmall;

    @ManyToOne(fetch = FetchType.EAGER) // Eager can be omitted, default value
    @JoinColumn(name="locker_id") //owner of the relation
    private Locker locker;

    @OneToMany(mappedBy="cell", fetch = FetchType.EAGER, cascade = CascadeType.REMOVE)
    private List<Parcel> parcels;

    // getters setters and constructors
    // . . .
}
```

Entity Parcel

```
@Entity
public class Parcel implements Serializable {
@Id @GeneratedValue(strategy = GenerationType.AUTO)
    private long parcelId;
    private String size;
    private String description;

    @Basic(fetch = FetchType.LAZY)
    @Lob
    private byte[] image;

    @ManyToOne(fetch = FetchType.EAGER) // Eager can be omitted, default
    @JoinColumn(name="cell_id") //owner of the relation
    private Cell cell;

    // getters setters and constructors
    // . . .
}
```